

▼ Consultas en MongoDB

- Ejercicio 1

▼ Consultas básicas en MongoDB

-  Ejercicio 2
-  Ejercicio 3
-  Ejercicio 4
-  Ejercicio 5
-  Ejercicio 6
-  Ejercicio 7
-  Ejercicio 8
-  Ejercicio 9
-  Ejercicio 10
-  Ejercicio 11
-  Ejercicio 12

▼ Consultas complejas en MongoDB

-  Ejercicio 13
-  Ejercicio 14
-  Ejercicio 15
-  Ejercicio 16
-  Ejercicio 17
-  Ejercicio 18
-  Ejercicio 19
-  Ejercicio 20

Consultas en MongoDB

La empresa **TechSolutions S.A.** ha migrado su sistema de gestión de empleados desde una base de datos relacional (MySQL) a MongoDB.

Actualmente se trabaja con la base de datos *empresa*, que contiene una tabla llamada *empleados*. Por cada empleado se tiene:

- `codigo` : identificador único del empleado
- `nombre` : nombre del empleado
- `dep` : departamento (Ventas, Compras o Facturación)
- `email` : correo electrónico corporativo
- `edad` : edad del empleado

El objetivo de esta práctica es traducir consultas SQL a su equivalente en MongoDB, utilizando los métodos de consulta y los operadores necesarios de filtrado, proyección y ordenación.

✓ Ejercicio 1

Inserta los siguientes datos en la colección `empleados` si en la base de datos relacional se hizo con la siguiente sentencia SQL:

```
INSERT INTO empleados (codigo, nombre, dep, email, edad)
VALUES
(1, 'Caresa', 'Ventas', 'scwilles@ning.com', 33),
(2, 'Malinde', 'Ventas', 'mabrahamowitz1@paypal.org', 30),
(3, 'Lauri', 'Compras', 'warhowd2@bluehost.com', 47),
(4, 'Richie', 'Facturación', 'roxlee3@chromengine.org', 45),
(5, 'Monti', 'Compras', 'mcraighei4@bandcamp.com', 31),
(6, 'Rosabel', 'Ventas', 'riverrills5@java.com', 26),
(7, 'Caresa', 'Ventas', 'cterransa6@bravesites.com', 53),
(8, 'Brendis', 'Facturación', 'burchfield7@cnbc.com', 37),
(9, 'Almeta', 'Compras', 'valow8@dyndns.org', 52),
(10, 'Berkeley', 'Ventas', 'etsbacy9@wiley.com', 33);
```

💡 Solución

```
db.empleados.insertMany([
  { codigo: 1, nombre: "Caresa", dep: "Ventas", email: "scwilles@ning.com", edad: 33 },
  { codigo: 2, nombre: "Malinde", dep: "Ventas", email: "mabrahamowitz1@paypal.org", edad: 30 },
  { codigo: 3, nombre: "Lauri", dep: "Compras", email: "warhowd2@bluehost.com", edad: 47 },
  { codigo: 4, nombre: "Richie", dep: "Facturación", email: "roxlee3@chromengine.org", edad: 45 },
  { codigo: 5, nombre: "Monti", dep: "Compras", email: "mcraighei4@bandcamp.com", edad: 31 },
  { codigo: 6, nombre: "Rosabel", dep: "Ventas", email: "riverrills5@java.com", edad: 26 },
  { codigo: 7, nombre: "Caresa", dep: "Ventas", email: "cterransa6@bravesites.com", edad: 53 },
  { codigo: 8, nombre: "Brendis", dep: "Facturación", email: "burchfield7@cnbc.com", edad: 37 },
  { codigo: 9, nombre: "Almeta", dep: "Compras", email: "valow8@dyndns.org", edad: 52 },
  { codigo: 10, nombre: "Berkeley", dep: "Ventas", email: "etsbacy9@wiley.com", edad: 33 }
]);
```

Consultas básicas en MongoDB

No se permite el uso de *Aggregation Pipeline* en esta serie de ejercicios.

✓ Ejercicio 2

Consulta equivalente a la siguiente sentencia SQL:

```
SELECT *  
FROM empleados  
WHERE codigo = 3;
```

💡 Solución

```
db.empleados.find(  
  { codigo: 3 }  
);
```

✓ Ejercicio 3

Consulta equivalente a la siguiente sentencia SQL pero sin mostrar el campo `_id`:

```
SELECT *  
FROM empleados;
```

💡 Solución

```
db.empleados.find(  
  {},  
  { _id: 0 }  
);
```

✓ Ejercicio 4

Consulta equivalente a la siguiente sentencia SQL:

```
SELECT nombre, email
FROM empleados
WHERE codigo = 3;
```



Solución

```
db.empleados.find(
  { codigo: 3 },
  { nombre: 1, email: 1 }
);
```

✓ Ejercicio 5

Consulta equivalente siguiente sentencia SQL pero sin mostrar el campo `_id` :

```
SELECT nombre, email
FROM empleados
ORDER BY nombre DESC;
```



Solución

```
db.empleados.find(
  {},
  { _id: 0, nombre: 1, email: 1 }
).sort({ nombre: -1 });
```

✓ Ejercicio 6

Consulta equivalente a la siguiente sentencia SQL:

```
SELECT *
FROM empleados
WHERE codigo IN (2,4,6);
```



Solución

```
db.empleados.find(  
  { codigo: { $in: [2, 4, 6] } }  
);
```

✓ Ejercicio 7

Consulta equivalente a la siguiente sentencia SQL sin mostrar el campo `_id`:

```
SELECT codigo, nombre  
FROM empleados  
WHERE codigo <> 5;
```

💡 Solución

```
db.empleados.find(  
  { codigo: { $ne: 5 } },  
  { _id: 0, codigo: 1, nombre: 1 }  
);
```

✓ Ejercicio 8

Consulta equivalente a la siguiente sentencia SQL:

```
SELECT *  
FROM empleados  
WHERE edad < 40 AND nombre='Rosabel';
```

💡 Solución

```
db.empleados.find(  
  { edad: { $lt: 40 }, nombre: "Rosabel" }  
);
```

✓ Ejercicio 9

Consulta equivalente a la siguiente sentencia SQL:

```
SELECT *  
FROM empleados  
WHERE codigo = 2 OR codigo = 4 OR codigo = 6;
```

💡 Solución

```
db.empleados.find(  
  { $or: [  
    { codigo: 2 },  
    { codigo: 4 },  
    { codigo: 6 }  
  ]}  
);
```

✓ Ejercicio 10

Realiza de 2 formas diferentes la Consulta equivalente a la siguiente sentencia SQL:

```
SELECT *  
FROM empleados  
WHERE email LIKE '%.org';
```

💡 Solución

Forma 1: utilizando `$regex`

```
db.empleados.find(  
  { email: { $regex: /\.org$/ } }  
);
```

Forma 2: sin usar `$regex`

```
db.empleados.find(  
  { email: /\.org$/ }  
);
```

✓ Ejercicio 11

Consulta equivalente a la siguiente sentencia SQL:

```
SELECT nombre, email  
FROM empleados  
WHERE dep = "Ventas"  
ORDER BY edad DESC  
LIMIT 3;
```

💡 Solución

```
db.empleados.find(  
  { dep: "Ventas" },  
  { _id: 0, nombre: 1, email: 1 }  
)  
.sort({ edad: -1 }).limit(3);
```

✓ Ejercicio 12

Consulta equivalente a la siguiente sentencia SQL:

```
SELECT *  
FROM empleados  
LIMIT 5, 3;
```

💡 Solución

```
db.empleados.find().skip(5).limit(3);
```

Consultas complejas en MongoDB

En estas consultas se permite el uso de *Aggregation Pipeline* para resolver los ejercicios, aunque no es obligatorio.

✓ Ejercicio 13

Consulta equivalente a la siguiente sentencia SQL:

```
SELECT LEFT(nombre,1) as inicial, nombre, email
FROM empleados;
```

💡 Solución

```
db.empleados.aggregate([
  {
    $project: {
      _id: 0,
      inicial: { $substr: ["$nombre", 0, 1] },
      nombre: 1,
      email: 1
    }
  }
]);
```

✓ Ejercicio 14

Consulta equivalente a la siguiente sentencia SQL:

```
SELECT AVG(edad) AS edadMedia
FROM empleados;
```

💡 Solución


```

db.empleados.aggregate([
  {
    $group: {
      _id: null,
      edadMedia: { $avg: "$edad" }
    }
  },
  {
    $project: {
      _id: 0,
      edadMedia: 1
    }
  }
]);

```

Ejercicio 15

Consulta equivalente a la siguiente sentencia SQL:

```

SELECT dep, COUNT(*) AS numEmpleados
FROM empleados
GROUP BY dep;

```

Solución

```

db.empleados.aggregate([
  {
    $group: {
      _id: "$dep",
      numEmpleados: { $sum: 1 }
    }
  },
  {
    $project: {
      _id: 0,
      dep: "$_id",
      numEmpleados: 1
    }
  }
]);

```

✓ Ejercicio 16

Consulta equivalente a la siguiente sentencia SQL:

```
SELECT dep, MAX(edad) AS mayor
FROM empleados
GROUP BY dep;
```

💡 Solución

```
db.empleados.aggregate([
  {
    $group: {
      _id: "$dep",
      mayor: { $max: "$edad" }
    }
  },
  {
    $project: {
      _id: 0,
      dep: "$_id",
      mayor: 1
    }
  }
]);
```

✓ Ejercicio 17

Consulta equivalente a la siguiente sentencia SQL:

```
SELECT dep, MAX(edad) AS mayor
FROM empleados
WHERE dep <> 'Ventas'
GROUP BY dep;
```

💡 Solución

```
db.empleados.aggregate([
  {
    $match: { dep: { $ne: "Ventas" } }
  },
  {
    $group: {
      _id: "$dep",
      mayor: { $max: "$edad" }
    }
  },
  {
    $project: {
      _id: 0,
      dep: "$_id",
      mayor: 1
    }
  }
]);
```

✓ Ejercicio 18

Consulta equivalente a la siguiente sentencia SQL:

```
SELECT dep, MAX(edad) AS mayor
FROM empleados
GROUP BY dep
HAVING MAX(edad) > 50;
```



Solución

```
db.empleados.aggregate([
  {
    $group: {
      _id: "$dep",
      mayor: { $max: "$edad" }
    }
  },
  {
    $match: { mayor: { $gt: 50 } }
  },
  {
    $project: {
      _id: 0,
      dep: "$_id",
      mayor: 1
    }
  }
]);
```

✓ Ejercicio 19

Consulta equivalente a la siguiente sentencia SQL:

```
SELECT dep
FROM empleados
WHERE dep<>'Ventas'
GROUP BY dep
HAVING MAX(edad)>50;
```



Solución

```
db.empleados.aggregate([
  {
    $match: { dep: { $ne: "Ventas" } }
  },
  {
    $group: {
      _id: "$dep",
      mayor: { $max: "$edad" }
    }
  },
  {
    $match: { mayor: { $gt: 50 } }
  },
  {
    $project: {
      _id: 0,
      dep: "$_id"
    }
  }
]);
```

✓ Ejercicio 20

Consulta equivalente a la siguiente sentencia SQL:

```
SELECT dep, SUM(edad) AS edadTotal
FROM empleados
WHERE edad < 40
GROUP BY dep
HAVING MAX(edad) < 50
ORDER BY edadTotal DESC;
```



Solución

```
db.empleados.aggregate([
  {
    $match: { edad: { $lt: 40 } }
  },
  {
    $group: {
      _id: "$dep",
      edadTotal: { $sum: "$edad" },
      mayor: { $max: "$edad" }
    }
  },
  {
    $match: { mayor: { $lt: 50 } }
  },
  {
    $project: {
      _id: 0,
      dep: "$_id",
      edadTotal: 1
    }
  },
  {
    $sort: { edadTotal: -1 }
  }
]);
```